Object Orientation

Conceptual Modeling

Encapsulation: Keep class variables privates to protect the encapsulation principle

Aggregation over Inheritance: Prefer aggregation because it is more flexible and relationships can change at runtime

Interfaces and Abstraction: Program to an interface or abstraction to reduce accidental complexity and unnecessary dependencies.

Loosely Coupled Layers: Organize software into layers (User interface, Application Logic, Persistent, Communications, Operating System) where each layer provides well-defined abstractions.

Naming Conventions: Use Pascal Casing nouns for classes, lower camel Casing for variables, and verb phrases for methods to improve readability.

Conceptual Modeling and UML: Use conceptual models at any stage to visualize systems and communicate efficiently. UML includes Use Case, Class, Interaction and State diagrams.

Unit testing: Use executable unit test cases with testing frameworks to find design errors, verify correctness and enable regression testing

Introduction to the Unified Modeling Language

What is UML

It is an object-oriented modeling language (or more precisely, a collection of modeling languages)

Key characteristics: Expressive, semi-formal, process-independent (can be used with various software development process models), and customizable. It allows elements to be hidden, left incomplete, or have inconsistencies

UML Building Blocks

Modeling elements are like words, diagrams are like sentences or paragraphs

Modeling Elements.

- Structural: Objects, classes
- Behavioral: Messages, states
- Organizational: Packages and views
- Annotational: Comments and specifications
- Extension: Templates and stereotypes.

Three Software-Engineering Modeling Perspectives.

1. Analysis (for understanding): Focuses on real-world objects and problem-domain concepts.

2. Specification (for solving problems and planning): Includes both real-world and software objects. Describes what the system is to become.

3. Implementation (for construction): Focuses mostly on software objects and solution-oriented concepts (libraries, frameworks, components).

Types of UML Diagrams

- Structural Diagrams (static view)

  Class Diagrams: show classes
  Object Diagrams: show object
  Component Diagrams: Illustrate static implementation views of a system
  Deployment Diagrams: show physical nodes

- Behavioral / Dynamic Diagrams

  Use case diagrams
  Sequence diagrams
  Collaboration
  Statechart
  Activity

Hints and Tips.

The main purpose is communication: To visualize, specify, construct. to communicate.
Keep a balance between structural and behavioral diagrams.

* Basic Use Cases

What are Actors and Use cases

Actors: External entities that need to interact with the system being built, such as users, other software systems, or hardware.

Use cases: Ideas representing what we is need to accomplish, independent of specific software features.

Use Cases vs System Interactions:

Use cases help answer "what" and "why" questions. focusing on user goals.

System interactions help answer "How" questions from a users perspective.

Use Case Diagram Relationships.

Actor Relationships (Generalization / Specialization). Cause actors are classifiers (sets of instances).

Extends (<<extends>>) = Defines a use case that is a variation or augmentation of another under specific conditions.

Includes (<<include>>) Defines how one use case requires the goal defined by another.

Generalization

Structure of a Use case diagram

A use case diagram consists of four primary components: Actors, Use Cases, Relationships, and Subjects.
They can also include notes, constraints, and packages.

Benefits

They capture user-visible functions and help identify who needs the system functionality.

Relationships between use cases document opportunities for reuse.

* Objetos, clases y Diagramas de Clases Básicos

Introduction to Objects and Classes

Objects: Fundamental building blocks of object-oriented systems. They represent any "thing", have boundaries in space and time, encapsulate state and behavior, and possess a unique identity

Classes: Abstractions that allow managing entire collections of objects sharing common properties and behaviors.

Three Modeling Perspectives

UML elements and diagrams are interpreted differently depending on the chosen perspective:

Analysis: Focuses on describing the problem domain. classes represent sets of real-world objects, and associations represent links between them.

Specification: Provides a logical description of the software system; classes represent interfaces or obligations to provide services/data.

Implementation (Design): Describes actual software components and deployment; classes represent abstract data types or code constructs.

3 Syntax for classes and Attributes

Class Structure: Class Name, Attributes, Operations, Responsibilities, and user-defined compartments.
Class names must be singular nouns or noun phrases using UpperCamelCase.

[visibility] name [multiplicity] [: type] [= initial-value]

[{ property-string }]

Visibility + (public), # (protected) - (private)

Attribute names use lowerCamelCase.

Types of Relationships in Class Diagrams.

- Generalization
- Association
- Aggregation and Composition
- Dependency.